

APACHE DORIS 3.0.8

复习题答案

第二章：数据组织、写入与表设计

12 道综合复习题的参考答案。每题包含一句话结论、底层因果链、得分关键词与常见误区。

配套资料：notes-02

基准版本：Apache Doris 3.0.8

整理日期：2026-08

答案导航

建议先在不看答案的情况下说出结论，再用本册检查自己能否解释完整因果链。能背出名词不等于真正理解了数据为什么这样组织。

每题按 10 分自测：结论准确 2 分；核心机制 4 分；因果链 2 分；能指出边界、误区或排查方法 2 分。

题号	核心问题	应答关键词
1	Partition 与 Bucket 为什么不能互换	范围、生命周期、Hash、Tablet
2	Tablet 与 Replica 数量怎样计算	各分区桶数之和、副本倍数
3	一个副本失败为何仍能提交	多数派、版本、修复
4	Rowset、Segment、Page 的分工	版本集合、列式文件、读取单元
5	Compaction 为何又快又会争资源	读放大、写放大、后台重写
6	三类索引分别适合什么查询	Prefix、Bloom、Inverted
7	三种数据模型如何处理同 Key	保留、聚合、覆盖
8	乱序 CDC 为何需要 Sequence	业务版本、LSN、状态倒退
9	COMMITTED、VISIBLE 与超时	提交、发布、Label、禁止盲目重发
10	查询为何固定读取 V10	READ COMMITTED、语句快照、MVCC
11	Aggregate 为何不能重复吃快照	SUM、增量、补偿、重算
12	慢查询如何逐层排查	分区、桶、索引、文件、执行计划

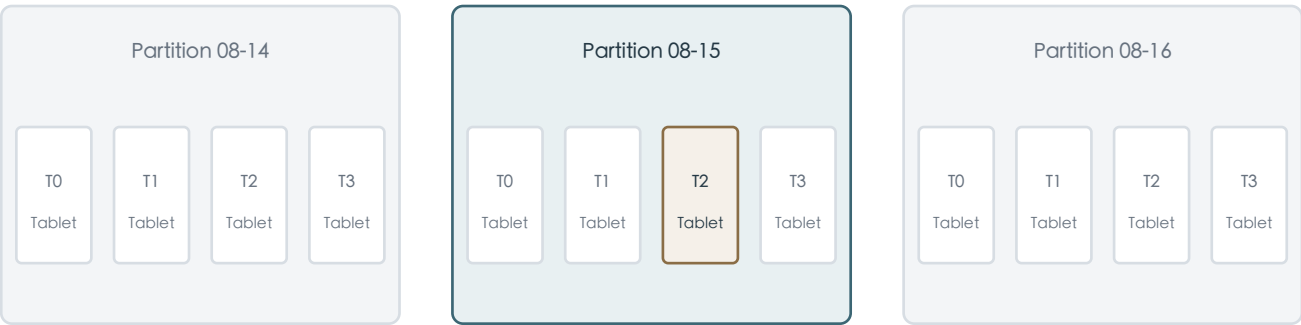
答题方法

- 先说明它解决什么问题，再说明它具体怎样工作。
- 把性能问题拆成少读、少传输、少计算，以及是否产生过多后台整理成本。
- 涉及更新正确性时，要同时检查完整 Key、Sequence 和事务可见状态。
- 排障时从最上层的裁剪开始，逐步进入 Tablet、Rowset、Segment 和执行算子。

1. Partition 和 Bucket 分别解决什么问题？为什么不能互相替代？

Partition 按范围划分数据，解决裁剪、生命周期和范围运维；Bucket 在每个分区内部按 Hash 形成 Tablet，解决均匀分布、并行执行和等值剪枝。

先按日期找分区，再按 Hash 找 Tablet



WHERE order_date='08-15' AND order_id=1001 只需进入中间分区的一个目标 Tablet

维度	Partition	Bucket / Tablet
切分方式	Range 或 List 等业务范围	对分桶键做 Hash
主要目的	范围裁剪、过期删除、回补与冷热管理	把分区分散到集群并提供扫描并行度
典型字段	日期、月份、有限业务区域	订单 ID、用户 ID 等高基数稳定字段
查询收益	先排除整块历史范围	在目标分区内继续定位少量 Tablet
物理结果	分区仍可很大	每个桶对应一个 Tablet 分片

完整因果链

```
WHERE order_date = '2026-08-15'  
AND order_id = 1001
```

date range -> partition pruning
hash(order_id) -> tablet pruning

日期先把其他月份或日期分区全部排除；order_id 再在命中的分区里计算 Hash，只访问目标 Tablet。没有日期条件时，Bucket 不能替你排除历史分区；只有分区没有分桶时，一个巨大分区又缺少足够的并行分片。

得分关键词	Partition=范围与生命周期；Bucket=Hash 规则；Tablet=实际分片；先分区裁剪再桶剪枝；二者正交
-------	---

常见误区：把 Bucket 当成全表范围。桶是在每个分区内部重复存在的；同一个 order_id 在不同分区会进入各自分区中的目标 Tablet。

2. 为什么基础 Tablet 数等于各分区桶数之和，而 Replica 数还要乘副本数？

每个分区独立拥有自己的桶和 Tablet；Replica 是每个 Tablet 的物理冗余实例，所以先把所有分区的桶数相加得到基础 Tablet 数，再乘副本数得到 Replica 数。

计算公式

```
base_tablets = sum(bucket_count_of_each_partition)
replicas     = base_tablets * replication_num
```

假设 12 个按月分区，每个分区 32 桶，三副本：

```
base_tablets = 12 * 32 = 384
replicas     = 384 * 3 = 1152
```

为什么不能说全表只有 32 个 Tablet

DISTRIBUTED BY HASH(order_id) BUCKETS 32 描述的是每个分区内部的切分规则。每增加一个同样规则的新分区，通常就增加另外 32 个基础 Tablet；自动分桶或不同分区采用不同桶数时，应对实际桶数求和。

Tablet 不是文件数量

- 一个 Tablet 会随着多次导入拥有多个 Rowset。
- 一个 Rowset 又可能包含多个 Segment 文件。
- 因此 Replica 数、Rowset 数和 Segment 文件数是三个不同维度。
- Rollup 或其他物化索引还会建立自己的 Tablet，本题公式只讨论 Base Index。

得分关键词

分区独立分桶；Tablet 数=各分区桶数之和；Replica 数=Tablet 数乘副本数；Tablet 不等于文件

常见误区：把三副本理解成逻辑行数变成三倍。SQL 仍看到一份业务数据；增加的是物理存储与写入副本。

3. 三副本中一个副本写入超时，事务为什么仍可能提交？之后如何恢复？

三副本达到两个成功副本后已满足多数派，事务可以进入提交与发布流程；失败副本被标记为异常或缺版本，后台随后从健康副本克隆或修复。

多数派让一个慢副本不必阻塞整个写入



为什么不要要求三份全部同时成功

1. 如果必须全部成功，最慢或故障节点会决定整个集群的写入可用性。
2. 只要求一个副本又太脆弱，该副本继续故障时可能失去可靠数据。
3. 多数派在可靠性、可用性和写入延迟之间取得工程折中。

提交后发生什么

- 健康副本完成版本发布，事务最终进入 `VISIBLE`。
- 落后副本不能因为进程存活就被当作完整可读副本，它必须拥有所需版本。
- 后台根据健康 `Replica` 克隆数据或补齐版本，恢复目标副本数。
- 修复会消耗磁盘、网络 and 后台线程，因此故障期可能伴随性能波动。

边界

多数派并不意味着任何一个副本失败都必然成功。如果失败数量超过提交要求、健康副本版本不完整，或 `Tablet` 本身处于异常状态，事务仍会失败或无法发布。

得分关键词

三副本多数派=2；提交与发布；落后版本不可读；健康副本克隆；后台修复；性能代价

常见误区：把“多数派提交”理解成失败副本可以永久不管。冗余下降后，下一次故障风险更高，后台必须尽快恢复副本健康。

4. Rowset、Segment、Page 分别承担什么职责？

`Rowset` 是一次导入或 `Compaction` 形成的版本集合；`Segment` 是 `Rowset` 中承载列数据和索引的不可变文件；`Page` 是 `Segment` 内更细的编码、压缩与按需读取单元。

从事务产物到磁盘读取粒度



对象	包含什么	为什么存在
Rowset	事务 ID、版本区间、多个 Segment	让 <code>Tablet</code> 以不可变版本追加和发布
Segment	各列数据、Footer、短 Key 与其他索引	形成可独立扫描、索引和合并的列式文件
Page	一小段列值、编码压缩数据、局部统计	只读取和解码需要的局部范围

为什么 Segment 不原地更新

- 新事务生成新 `Rowset`，不破坏正在读取旧快照的查询。
- 顺序写、批量编码和列压缩更符合分析负载。
- 事务提交、版本发布、故障恢复和 `Compaction` 都能围绕不可变对象工作。

- 代价是旧版本与小 Rowset 累积，需要后台合并和延迟回收。

Compaction 后语义为什么不变

$$[0-8] + [9-9] + [10-10] + [11-11] \\ == \\ [0-11]$$

物理读取路径减少了，但逻辑版本覆盖仍相同；旧查询持有旧 Rowset 引用时，旧文件也不会被立即破坏。

得分关键词

Rowset=版本集合；Segment=不可变列式文件；Page=编码读取单元；版本链；Compaction 物理重写不改逻辑结果

常见误区：把 Rowset 当作单个文件，或把 Page 当作一行。二者都是集合或批量组织，目的正是避免分析系统逐行处理。

5. Compaction 为什么既能改善查询，也可能与导入争抢资源？

Compaction 读取多个旧 Rowset、重新合并并写出更大的新 Rowset，用后台写放大换取更低的前台读放大；重写过程会占用 CPU、磁盘、内存和线程，因此可能与导入、查询竞争。

它怎样改善查询

many small rowsets
-> open many segments and indexes
-> merge more versions while reading

after compaction
-> fewer rowsets and segments
-> shorter read path

它为什么消耗资源

资源	Compaction 的消耗	对前台的可能影响
磁盘 I/O	读取旧 Segment 并写出新 Segment	导入 Flush 和查询扫描延迟上升
CPU	解码、比较、聚合、主键处理与编码	查询算子和导入处理可用 CPU 减少
内存	合并缓冲、元数据和索引构建	内存紧张时触发更频繁 Flush 或限流
后台线程	多个 Tablet 的合并任务	积压时版本增长速度超过整理速度

正确排查顺序

1. 先看写入是否过碎：逐行或极小批次会持续制造小 Rowset。
2. 定位是哪些 Tablet 的 Rowset、Segment 和 Compaction 得分异常，而不是只看平均值。
3. 检查 BE 磁盘空间、I/O、CPU、负载倾斜和后台任务。
4. 最后再调整批次、桶数、索引和 Compaction 参数，避免用参数掩盖模型问题。

得分关键词

读放大；后台写放大；读取旧文件并写新文件；CPU/I/O/内存竞争；微批；定位热点 Tablet

常见误区：把 Compaction 当成免费的清理线程。它能降低未来查询成本，但整理历史文件本身就是一次重型数据处理。

6. Prefix、Bloom Filter、Inverted Index 分别适合什么查询？

Prefix 适合符合排序 Key 最左前缀的等值和范围过滤；Bloom Filter 适合高基数等值存在性判断；Inverted Index 适合不符合前缀的灵活值查询、文本检索和部分复杂条件。

机制	适合	怎样减少扫描	关键限制
Prefix Index	Key 最左前缀等值或范围	利用排序局部性定位短 Key 范围	跳过前列后，后列通常不能继续有效定位
Bloom Filter	订单号、设备号等高基数等值条件	判断某块一定不存在目标值	有假阳性，不支持精确定位和范围查询
Inverted Index	文本、字符串、数值的灵活过滤	值或词项映射到命中行号集合	增加写入、磁盘和 Compaction 成本

例子

DUPLICATE KEY(event_date, merchant_id, event_time, order_id)

event_date = ? AND merchant_id = ? -> good prefix use
order_id = ? -> weak prefix use

order_id 单独查询不符合排序前缀，可以通过 HASH(order_id) 做桶剪枝，再用倒排索引在目标 Tablet 内定位。
payment_no 的高基数等值查询也可以评估 Bloom Filter，但是否有效必须用真实基数和扫描字节验证。

索引选择原则

- 先保证分区、分桶和 Key 顺序合理，再补充二级索引。
- 低选择性状态列即使有索引，也可能返回大量行号。
- 索引需要随导入和 Compaction 构建，不是越多越好。
- 通过 EXPLAIN、Query Profile、索引命中与扫描字节确认收益。

得分关键词	Prefix=排序最左前缀；Bloom=高基数等值与概率判断；Inverted=行号映射与灵活条件；索引有写放大
-------	--

常见误区：Bloom Filter 说“可能不存在”是不准确的：判断不存在时一定不存在；判断可能存在时仍需读取验证。

7. Duplicate、Aggregate、Unique MOW 在同 Key 写入时有什么差异？

Duplicate 保留全部记录；Aggregate 按聚合函数合并 Value；Unique MOW 按完整主键维护一个最新有效版本，并在写入侧通过 Delete Bitmap 处理旧版本。

模型	同 Key 再写一条	典型数据	关键风险
Duplicate	新旧全部保留	日志、事件、明细事实、原始 CDC	误以为 Key 会自动去重
Aggregate	按 SUM、MAX、MIN 等合并 Value	商家日报、可加指标	把完整快照当增量导致重复累计
Unique MOW	新版本生效，旧版本在新快照中失效	订单当前状态、用户画像、CDC	完整 Key 不稳定或乱序覆盖

同一个订单的例子

10:00 order 1001 CREATED
10:02 order 1001 PAID
10:05 order 1001 SHIPPED

- Duplicate：三条全部保留，适合追踪完整事件历史。
- Unique MOW：当前快照只返回 SHIPPED，前提是 Key 和 Sequence 正确。
- Aggregate：只有把事件转换成合法指标增量后才适合写入，不能直接用状态字符串 SUM。

为什么这是语义选择

模型决定同 Key 数据最终代表什么。模型错误时，SQL 可能运行很快但业务答案仍然错误；因此必须先定义“一行是什么事实”，再讨论索引、分桶和 Compaction。

得分关键词

Duplicate=全保留；Aggregate=按函数合并；Unique MOW=最新有效版本；模型先于性能；完整 Key

常见误区：Duplicate Key 中的 Key 主要承担排序与索引前缀作用，不代表唯一约束。

8. 没有 Sequence Column 时，乱序 CDC 会造成什么问题？

如果只按数据到达 Doris 的顺序覆盖，迟到的旧事件可能把较新的业务状态覆盖掉，造成订单状态倒退；Sequence Column 用源端业务版本决定谁更新，而不是谁最后到达。

到达顺序不等于业务发生顺序



Sequence 应选择什么

候选	优点	风险
源数据库 LSN / binlog position	与源端变更顺序一致，通常最可靠	需要正确映射类型和全局/分片语义
业务递增版本号	语义明确，可跨链路传递	上游必须保证单调性
update_time	容易获得和理解	时间精度相同、时钟或回填可能产生歧义
Doris 到达时间	实现简单	无法解决网络延迟与乱序，通常不合适

还必须保证完整 Key 稳定

Sequence 只能在同一个 Unique Key 内比较版本。如果 create_date、merchant_id 或其他 Key 列在更新时变化，Doris 会认为它是另一行，Sequence 无法把两个 Key 合并。

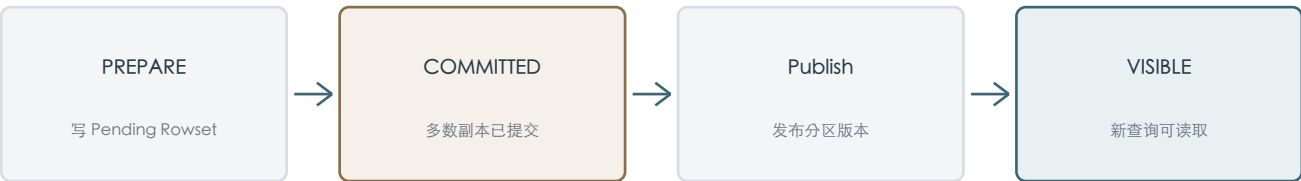
得分关键词	到达顺序不可靠；旧事件覆盖新状态；Sequence 使用源业务版本；优先 LSN；完整 Unique Key 必须稳定
-------	---

常见误区：配置 Sequence 后就可以忽略 Key。Sequence 只解决同 Key 的版本先后，不解决 Key 设计错误和跨分区主键变化。

9. COMMITTED 和 VISIBLE 有什么区别？Publish Timeout 后应该怎么做？

COMMITTED 表示事务已经可靠提交但版本可能尚未发布；VISIBLE 表示 Rowset 已进入可见版本链。Publish Timeout 后应使用原 Label 查询状态，COMMITTED 继续等待，绝不能直接换新 Label 重发。

一次导入从写入到查询可见



COMMITTED 表示不能再正常回滚；VISIBLE 才表示查询可见

状态	含义	客户端动作
PREPARE	正在准备和写 Pending Rowset	等待或根据导入结果排错
PRECOMMITTED	2PC 等待外部 commit/abort	由外部 Checkpoint 决定
COMMITTED	多数副本已提交，尚未保证查询可见	轮询状态，不重发
VISIBLE	版本发布完成，新查询可以读取	确认成功并推进上游进度
ABORTED	事务失败，数据不进入可见版本	修复原因后发起新的可审计尝试

为什么换 Label 重发危险

第一次事务可能已经 COMMITTED，只是客户端没有及时看到 VISIBLE。新 Label 会被认为是另一批合法数据：Duplicate 可能产生重复行，Aggregate 可能重复 SUM，甚至让下游指标静默翻倍。

```
SHOW TRANSACTION FROM order_db
WHERE LABEL = 'orders_kafka_p3_1000_1999';
```

Label 的设计

使用可重建的来源标识，例如 topic、partition 和 offset 范围。相同源批次始终得到相同 Label，网络重试时 Doris 才能识别它不是一批新数据。

得分关键词

COMMITTED=可靠提交未必可读；VISIBLE=发布完成；Publish Timeout 不等于失败；原 Label 查询；禁止盲目重发

常见误区：只看 HTTP 或客户端超时就判断导入失败。分布式导入必须把请求结果与服务端事务状态分开判断。

10. 为什么查询 Q1 在运行中遇到 V11 发布，仍继续读取 V10？

Doris 在每条查询语句开始时捕获一致快照。Q1 已绑定 V10，执行期间新事务变为 V11 不会改变它的读取集合；之后启动的 Q2 才捕获 V11。

语句级 READ COMMITTED 快照



如果查询中途切换版本会怎样

一个聚合可能前半段按旧订单状态统计，后半段按新状态统计，得到无法对应任何真实时刻的结果。稳定快照保证一次语句内部的数据世界不变化。

Compaction 为什么也不能改变 Q1

Q1 path : [0-8] + [9] + [10]
new path: [0-10] after compaction

Compaction 可以生成新的物理读取路径，但旧 Rowset 在仍有查询引用时不会立即删除。无论 Q1 走旧路径还是后续查询走新路径，逻辑版本 V10 的结果必须相同。

Unique MOW 的旧行如何处理

V11 的 Delete Bitmap 只让旧记录从 V11 快照开始失效。读取 V10 的 Q1 仍能看到旧状态，读取 V11 的 Q2 才跳过旧记录并读取新状态。

得分关键词

READ COMMITTED；语句开始捕获快照；Q1=V10；Q2=V11；Compaction 只改物理路径；Delete Bitmap 按版本生效

常见误区：把 READ COMMITTED 理解成查询每读取一页都重新取最新版本。隔离级别是语句级快照语义，不是持续追赶最新数据。

11. 为什么 Aggregate 表不能直接重复接收当前状态快照？

Aggregate 的 SUM 会忠实合并每次输入值，它不知道输入是新增量还是同一业务对象的重复快照；完整金额快照重复写入会被重复累计。

错误示例

```
snapshot 1: order 1001, paid_gmv = 100
snapshot 2: order 1001, paid_gmv = 100
```

```
SUM result = 200 # engine is correct, business input is wrong
```

正确输入应是变化量

```
first payment      -> order_count +1, gmv +100
payment cancelled  -> order_count -1, gmv -100
amount 100 to 120  -> order_count  0, gmv  +20
```

三种可靠方案

1. 上游根据状态转换生成正负增量，并用事件 ID 或 Checkpoint 保证增量只成功一次。
2. 先在 Unique 当前状态表中去重，再由可靠任务计算目标分区，但不要假设同一显式事务内可读刚写入数据。
3. 对于难以生成补偿增量的指标，按日期重算并替换整个 ADS 分区。

什么时候 Aggregate 合理

只有当合并函数与业务语义一致时才合理。例如可加金额使用 SUM，最后更新时间使用 MAX。若指标依赖复杂去重或不可逆状态转换，应在上游或明细层先计算。

得分关键词

Aggregate 不识别快照；SUM 重复累计；输入必须是增量；正负补偿；幂等事件；分区重算

常见误区：认为 Aggregate Key 会根据 order_id 自动去重。只有 Aggregate Key 中定义的 Key 决定聚合粒度，Value 会按指定函数合并。

12. 面对慢查询，怎样按裁剪与存储层次逐步排查？

从最便宜、影响最大的范围裁剪开始：Partition -> Tablet -> Key/索引 -> Segment/Page/列 -> Rowset/Compaction -> Join/Shuffle/聚合。先确认读了多少数据，再讨论计算为什么慢。

慢查询排查漏斗



第一层：计划到底扫描了什么

1. 用 EXPLAIN 查看分区数量，确认时间谓词能直接推导到分区列。
2. 检查等值条件是否包含分桶键，目标分区内是否发生 Tablet 剪枝。
3. 确认 Key 顺序是否匹配过滤条件，Prefix、Zone Map、Bloom 或 Inverted Index 是否命中。

第二层：实际读取了多少

- Query Profile 中扫描行数、扫描字节和过滤后行数是否远高于预期。
- 是否只读取需要的列，还是 SELECT * 把宽表大量列送入执行引擎。
- 高选择性谓词是否足够早地下推到 Scan。
- 热点 Tablet 是否数据倾斜，导致少数 BE 扫描远多于其他节点。

第三层：存储是否碎片化

- 同一 Tablet 的 Rowset 和 Segment 是否过多，Compaction 是否长期积压。
- 微批是否太小、索引是否过多、磁盘空间和 I/O 是否不足。
- Unique MOW 查询是否需要处理大量版本或 Delete Bitmap，副本是否健康。

第四层：执行算子与网络

- Join 顺序、Build 侧大小、Runtime Filter 到达时间和命中率是否合理。
- Shuffle 数据量、数据倾斜、Broadcast 大小与聚合前后行数是否异常。
- 并发、内存限制、Spill、CPU 和 BE 负载是否造成排队。

一个实用判断

如果扫描字节已经很少但查询仍慢，重点转向 Join、Shuffle、聚合和排队；如果扫描字节巨大，应先修复分区、分桶、Key 或索引。不要在读取范围错误时先调整线程参数。

得分关键词

EXPLAIN；分区裁剪；桶剪枝；Key/索引命中；扫描字节；Rowset/Compaction；倾斜；Join/Shuffle；Query Profile

常见误区：一看到慢查询就增加倒排索引。若根因是未带时间条件、Join Shuffle 或 Compaction 积压，新索引只会增加写入成本。

答案速记

题号	一句话记忆
1	Partition 切范围，Bucket 做 Hash，Tablet 承载实际分片。
2	先加总各分区桶数得到 Tablet，再乘副本数得到 Replica。
3	三副本两个成功可达多数派，落后副本后续修复。
4	Rowset 管版本集合，Segment 管列式文件，Page 管局部读取。
5	Compaction 用后台写放大换前台低读放大。
6	Prefix 看最左排序前缀，Bloom 看等值不存在，Inverted 找行号。
7	Duplicate 全保留，Aggregate 按函数合并，Unique MOW 留最新。
8	Sequence 以业务版本抵抗 CDC 乱序，不能代替稳定 Key。
9	COMMITTED 只是已提交，VISIBLE 才可读；超时用原 Label 查。
10	Q1 固定 V10，V11 只给之后启动的查询。
11	Aggregate 要吃增量，重复快照会被重复 SUM。
12	慢查询先看少没少读，再看 Join、Shuffle 和资源。

自测分数解释

总分	掌握程度	下一步
100-120	能够从业务语义推导到物理设计	进入第三章，开始导入链路与实践
80-99	主体清楚，部分版本/索引边界不牢	重看 2.6-2.10 并结合 EXPLAIN 演练
60-79	记住了层级，但因果链仍不完整	重画 Table 到 Page 与事务可见性图
0-59	概念容易混用	从 2.1 开始按一行数据的路径重新学习

参考资料

答案依据第二章教程整理，并以 Apache Doris 3.0 与 3.x 官方文档中的稳定机制为校验来源。

[1] Basic Concepts

<https://doris.apache.org/docs/3.x/table-design/data-partitioning/basic-concepts/>

[2] Data Bucketing

<https://doris.apache.org/docs/3.x/table-design/data-partitioning/data-bucketing/>

[3] Load High Availability

<https://doris.apache.org/docs/3.x/data-operate/import/load-high-availability/>

[4] Rowsets System Table

https://doris.apache.org/docs/3.0/admin-manual/system-tables/information_schema/rowsets

[5] Compaction Troubleshooting

<https://doris.apache.org/docs/3.x/admin-manual/trouble-shooting/compaction/>

[6] Prefix Index

<https://doris.apache.org/docs/3.x/table-design/index/prefix-index/>

[7] Bloom Filter Index

<https://doris.apache.org/docs/3.x/table-design/index/bloomfilter/>

[8] Inverted Index

<https://doris.apache.org/docs/table-design/index/inverted-index>

- [9] Data Model Overview
<https://doris.apache.org/docs/3.x/table-design/data-model/overview/>
- [10] Unique Update Concurrent Control
<https://doris.apache.org/docs/3.0/data-operate/update/unique-update-concurrent-control/>
- [11] Transaction
<https://doris.apache.org/docs/3.0/data-operate/transaction/>
- [12] SHOW TRANSACTION
<https://doris.apache.org/docs/3.0/sql-manual/sql-statements/transaction/SHOW-TRANSACTION>
- [13] Stream Load Principle
<https://doris.apache.org/blog/principle-of-Doris-Stream-Load/>
- [14] Group Commit
<https://doris.apache.org/docs/3.x/data-operate/import/group-commit-manual/>
- [15] Release 3.0.8
<https://doris.apache.org/releases/v3.0/release-3.0.8/>